

Artefato SBSeg 2026 — Apêndice

Artigo #189 — TopVenues: An Executable Corpus and Research Tool for Cybersecurity Literature Reviews

Sidnei Barbieri, Ágney Roth Ferraz, Lourenço Alves Pereira Júnior

¹Instituto Tecnológico de Aeronáutica (ITA)
São José dos Campos — SP — Brasil

sidneisb@ita.br

Resumo. *Revisões de literatura em segurança dependem de um denominador estável: o conjunto de artigos elegíveis antes de a triagem começar. Na prática, esse denominador é remontado a partir de portais, APIs e exportações que mudam ao longo do tempo, o que torna o corpus resultante difícil de auditar ou reutilizar. O TopVenues transforma a construção do corpus em um artefato de pesquisa executável: dado um escopo declarado de veículos e anos, normaliza metadados do DBLP, enriquece registros com resumos e entradas BibTeX, e materializa um snapshot SQLite monotônico exposto por linha de comando, aplicação web, busca ranqueada e exportações orientadas a revisão.*

1. Selos Considerados

Os selos considerados são: **Disponíveis (SeloD)**, **Funcionais (SeloF)**, **Sustentáveis (SeloS)** e **Experimentos Reprodutíveis (SeloR)**.

2. Informações básicas

Acesso ao artefato. O artefato é público, sem credenciais, em repositório estável no GitHub:

<https://github.com/sidneibarbieri/topvenues-tool>

A versão avaliada é a v1.9.3. O README.md do repositório é a documentação normativa e segue o modelo mínimo do CTA.

Não são necessários recursos externos. Não há infraestrutura de nuvem, chave SSH, chave de API, credencial, conta de serviço ou máquina virtual fornecida pelos autores. O snapshot do corpus acompanha o repositório e a avaliação é integralmente local.

Ambiente usado nos experimentos relatados no artigo.

Sistema operacional	macOS 26.5.2 (build 25F84)
Kernel	Darwin 25.5.0
Arquitetura	arm64 (Apple Silicon)
Processador	Apple M4 Max, 16 núcleos
Memória RAM	64 GB
Armazenamento livre	53 GB
Python	3.14.7

Requisitos mínimos para reprodução. O artefato não exige o hardware acima. Foram verificados: Linux, macOS ou Windows 10+; Python 3.11 ou superior (testado em 3.11, 3.12, 3.13 e 3.14); 4 GB de RAM (pico observado inferior a 1 GB); 3 GB de disco; rede apenas durante a instalação das dependências; navegador atual apenas para a interface web; usuário comum, sem privilégios de administrador.

Tempo esperado. Instalação das dependências entre 60 e 120 segundos; reprodução completa em cerca de 4 minutos no ambiente descrito, e até 12 minutos em máquina modesta.

2.1. Dependências

Todas as dependências de execução são instaladas a partir de `requirements-frozen.txt`, que fixa 71 pacotes por versão exata e contém 1.161 hashes SHA-256. A instalação usa `pip install --require-hashes`, de modo que qualquer divergência de bytes interrompe a instalação em vez de produzir um ambiente diferente do relatado. O mesmo conjunto está disponível como `uv.lock` para o gerenciador `uv`, que o script de reprodução prefere quando o encontra.

As dependências da interface web estão nesse mesmo arquivo (`streamlit`, `altair`, `watchdog`). Não existe passo adicional de instalação: o script exibe essa informação explicitamente na primeira etapa. O arquivo `requirements-web.txt` é apenas uma declaração legível do subconjunto web e não é usado por nenhum caminho de instalação.

Principais versões fixadas: `pandas 3.0.3`, `pyarrow 24.0.0`, `pydantic 2.13.4`, `httpx 0.28.1`, `beautifulsoup4 4.15.0`, `click 8.4.2`, `rich 15.0.0`, `pyyaml 6.0.3`, `streamlit 1.58.0`, `altair 6.2.2`, `pytest 9.1.1` e `ruff 0.15.20`. A lista completa, com hashes, está no repositório.

Recursos de terceiros. Nenhum é necessário para a reprodução. Os serviços externos (DBLP, Semantic Scholar, OpenAlex, CrossRef, ACM Digital Library, IEEE Xplore, USENIX e NDSS) são usados apenas pelos comandos opcionais de coleta, que não fazem parte da reprodução e não alteram o snapshot publicado. Todos são consultados por endpoints públicos e sem autenticação.

2.2. Preocupações com segurança

A execução do artefato não oferece risco ao avaliador. O artefato não requer privilégios de administrador; não instala serviços, não altera configurações do sistema e não abre portas externas (a interface web escuta apenas em `localhost`); não executa código de terceiros baixado em tempo de execução, pois as dependências são fixadas por hash; não coleta nem transmite dados pessoais do avaliador. Após a instalação das dependências, a reprodução é offline. Todos os arquivos criados ficam dentro do diretório clonado, e remover esse diretório desfaz completamente a instalação. O snapshot é aberto em modo somente leitura (`mode=ro&immutable=1`), de forma que a execução não pode corromper o corpus verificado.

3. Instalação

Linux e macOS:

```
git clone https://github.com/sidneibarbieri/topvenues-tool
cd topvenues-tool
bash reproduce.sh --profile security-20
```

Windows (PowerShell):

```
git clone https://github.com/sidneibarbieri/topvenues-tool
cd topvenues-tool
powershell -ExecutionPolicy Bypass -File .\reproduce.ps1 -Profile secur
```

Alternativa por contêiner:

```
docker compose up
```

O script cria um ambiente virtual isolado, instala as dependências fixadas, materializa o snapshot, verifica seu SHA-256 contra o manifesto, executa a suíte de testes, renderiza a interface, confere as reivindicações do artigo, compara a busca ranqueada com a linha de base e exporta uma amostra BibTeX. Ao final desse comando a ferramenta está instalada e verificada.

Observação sobre o Windows. Se a interface web estiver aberta, o sistema operacional mantém o arquivo do banco bloqueado e a materialização falha com `CorpusBusyError`. Feche o Streamlit e execute o script novamente. A materialização é protegida por um lock atômico e multiplataforma, de modo que duas execuções simultâneas se serializam em vez de corromper o banco.

4. Teste mínimo

A instalação cria um ambiente virtual em `.venv/`. Ative-o uma vez por sessão de terminal antes dos comandos abaixo — `source .venv/bin/activate` no Linux e no macOS, `.\.venv\Scripts\Activate.ps1` no Windows — ou chame o interpretador pelo caminho direto, `.venv/bin/python`. Sem isso, `python` pode não existir no sistema ou ser uma versão antiga sem as dependências.

```
python -m src.cli --profile security-20 stats
python -m src.cli --profile security-20 search --title "intrusion detec
python -m src.cli --profile security-20 search --rank "memory corruptio
python -m src.cli --profile security-20 export --title intrusion \
    --format bibtex --output amostra.bib
python -m streamlit run web/app.py
```

Resultado esperado: o primeiro comando imprime `Total Papers: 20305 e With Abstracts: 17491 (86.14%)`; o segundo exibe 50 registros (limite padrão, de 157 correspondências); o terceiro retorna registros ordenados por relevância BM25; o quarto grava um arquivo com 2.928 linhas; o quinto abre a interface em `http://localhost:8501` com cinco páginas navegáveis. Tempo total aproximado: 90 segundos.

5. Experimentos

Todas as reivindicações abaixo são verificadas por um único comando, que também é executado dentro dos scripts de reprodução:

```
python scripts/verify_paper_claims.py --profile security-20
```

Recursos comuns a todos os experimentos: menos de 1 GB de RAM, menos de 3 GB de disco, sem rede e sem GPU.

5.1. Reivindicação #1 — 20.305 artigos entre 2017 e 2026

Seção 4 do artigo. Tempo: 20 s. Esperado: Claim #1 ... observed 20305 e Claim #2 ... observed '2017-2026'.

5.2. Reivindicação #2 — Escopo declarado de 20 veículos

Seção 3 do artigo. O escopo é dado por `config.yaml`, fora do código. Tempo: 20 s. Esperado: Claim #3 ... observed 20.

5.3. Reivindicação #3 — 17.491 registros com resumo (86,1%)

Seção 4 do artigo. Tempo: 20 s. Esperado: Claim #4 ... observed 17491 e Claim #5 ... observed 86.1.

5.4. Reivindicação #4 — Todo registro carrega entrada BibTeX

Seção 5 do artigo. Tempo: 30 s. Esperado: Claim #6 ... observed 20305 e um arquivo BibTeX com 2.928 linhas.

5.5. Reivindicação #5 — Identidade verificável offline por SHA-256

Seção 6 do artigo. Comando: `python scripts/verify_profile_snapshot.py --profile security-20`. Tempo: 15 s. Esperado: o hash confere com o manifesto (5a35bd6e...7493b08).

5.6. Reivindicação #6 — Busca ranqueada frente à linha de base

Seção 5 do artigo. Este experimento traz o **referencial de comparação**: a busca por substring (LIKE), equivalente ao que uma planilha ou um `grep` fazem, medida contra a busca ranqueada FTS5/BM25 sobre o mesmo corpus e na mesma máquina. Comando: `python scripts/benchmark_search.py --profile security-20 --trials 11`. Tempo: 40 s.

Consulta	Linha de base LIKE	Ranqueada BM25	Ganho
machine learning	1.962 res., 37,6 ms	2.093 res., 27–32 ms	~1,3×
fuzzing	661 res., 36,0 ms	661 res., 6,1 ms	~5,9×
intrusion detection	337 res., 65,5 ms	354 res., 3,9 ms	~16,8×
ransomware	84 res., 55,8 ms	84 res., 1,5 ms	~37×

As contagens são determinísticas e se repetem a cada execução; os tempos são medianas de 11 repetições e variam entre máquinas. O que o experimento demonstra é a relação entre as duas colunas.

5.7. Reivindicação #7 — Suíte de testes executada offline

Seção 6 do artigo. Comando: `python -m pytest -q`. Tempo: 10 s. O artigo submetido informa 238 testes, número correto na data da submissão; desde então o artefato recebeu correções e a suíte cresceu, sem que nenhum teste fosse removido. As contagens do corpus permanecem idênticas às do artigo, como as reivindicações #1 a #4 demonstram. O número corrente é declarado no `README.md` e verificado automaticamente.

6. Verificação contínua

A cada alteração publicada, o GitHub Actions executa a reprodução completa em oito ambientes: Ubuntu e Windows, cada um com Python 3.11, 3.12, 3.13 e 3.14. Em cada ambiente os dois perfis são reproduzidos, incluindo o `security-20` citado no artigo. A reprodução em Windows, portanto, é verificada automaticamente e não depende de configuração manual. O registro de execução de cada ambiente fica disponível como artefato do fluxo de trabalho, e o estado corrente aparece no distintivo `tests` no topo do `README.md`.

7. Informações adicionais

Documentação. O `README.md` do repositório segue o modelo mínimo do CTA e contém todas as seções exigidas, incluindo solução de problemas para Windows. A documentação em inglês está em `README.en.md`.

Identificação das reivindicações no artefato. O script `scripts/verify_paper_claims.py` associa cada afirmação numérica do artigo à seção que a enuncia e à consulta que a verifica, e é executado dentro de `reproduce.sh` e `reproduce.ps1`.

Demonstração. Um vídeo de 7min49s em 1920x1080, com narração em inglês e legendas em português e inglês, acompanha o repositório em `docs/assets/demos/` e cobre instalação, verificação offline, busca, exportações e as análises da interface.

Dataset publicado. A exportação em Parquet do corpus está disponível em <https://huggingface.co/datasets/sidneibarbieri/topvenues>, com o identificador de perfil, o hash do snapshot e a versão de origem registrados no cartão do dataset.

Licença. Código sob licença MIT. Metadados bibliográficos e entradas BibTeX provenientes do DBLP, sob os termos CC0 do DBLP.